

SHERPA

Sports, Heritage and Excursion Recommendation and Planning Agent Application Code Guide (Pilot)

The application modules, data pipelines, agent orchestration and the exact Streamlit Cloud secrets

Author: Martín Galán

Repository: github.com/mgalanme/sherpa

Companion package: SHERPA_App_Scripts.zip

Version 0.1 (Draft for review) | Date: 13 June 2026

Table of Contents

- 1. Introduction 3
- 2. How the Application Fits Together 4
- 3. Package Layout 5
- 4. Module Reference 6
- 5. Data Pipelines 8
- 6. The Human-in-the-Loop Workflow in Code 9
- 7. Streamlit Community Cloud Deployment and Exact Secrets 10
- 8. Deployment and Run Order 12
- 9. Dependencies and Resource Notes 13
- 10. Limitations and Next Steps 14

1. Introduction

This document describes the SHERPA application code for the pilot. As with the previous deliverables, the document references the code but does not contain it. The code is delivered separately in SHERPA_App_Scripts.zip. This guide is the map that explains what each module does, how the pieces fit together, how the data pipelines feed the application, how the Human-in-the-Loop workflow is implemented, and the exact secrets required to run the application on Streamlit Community Cloud.

It builds directly on the Foundational Architecture Document and the Pilot Setup Guide. The environments, accounts and connectivity established by the Setup package are assumed to be in place and verified by the smoke tests.

Convention: module and file names, functions and variables appear in a monospace font. British English is used throughout, with no em dashes and no separator lines between paragraphs.

2. How the Application Fits Together

A request enters through the Streamlit portal, either as a free-text description that the intake layer structures with the language model, or through the template form. The structured inputs are geocoded and passed to the orchestration graph.

The orchestration is a LangGraph workflow that gathers weather, access, route and heritage data in parallel, then runs the classical recommendation engine, composes a draft dossier with a grounded narrative, and stops at a Human-in-the-Loop checkpoint. Throughout, each step publishes a governance and observability event to the Solace broker, so the event mesh genuinely runs in the cloud through Solace Cloud, or locally through PubSub+. The reviewer then approves, modifies or takes another decision, and on approval a PDF dossier is produced. Every action is written to the append-only, hash-chained audit trail.

Design stance reaffirmed: the language model only understands input and drafts narrative. The equipment checklist and the risk flags are produced by deterministic, explainable rules in `recommend.py`, so safety-relevant advice is reproducible and auditable.

3. Package Layout

Path	Contents
src/config.py	Settings from environment or Streamlit secrets
src/models.py	Pydantic v2 domain models (inputs, route, weather, dossier, decisions)
src/catalog.py	Activity taxonomy and the deterministic equipment rules
src/intake.py	Geocoding and the language-model parser for free-text intake
src/llm.py	Groq primary with Ollama local fallback
src/embeddings.py	In-process nomic embeddings (768 dims)
src/clients/	Open-data clients: weather, geo, route, culture, nature
src/stores/	Qdrant, Neo4j and Databricks access
src/recommend.py	Classical recommendation and risk engine
src/audit.py	Hash-chained audit trail (Delta or local fallback)
src/dossier.py	PDF rendering with ReportLab
src/mesh.py	Solace event publication for governance and observability
src/agents/	LangGraph orchestration (graph, nodes) and the narrative crew
src/portal/app.py	Streamlit portal and the HITL console
databricks/notebooks/	Medallion notebooks: bronze, silver, gold with embeddings
prompts/	The intake and narrative master prompts
.streamlit/secrets.toml.example	The exact Streamlit Cloud secrets
scripts/	Application deploy and run scripts (20, 21, 22)

4. Module Reference

Each module is summarised with its responsibility and its principal entry points. The lessons from earlier case studies are applied where relevant and noted.

Module	Responsibility and key entry points
<code>config.py</code>	<code>get_settings()</code> builds a cached Settings object from the environment. <code>load_streamlit_secrets()</code> copies Streamlit secrets into the environment so the same configuration works locally and in the cloud.
<code>models.py</code>	The shared contract: ActivityInputs, GeoPoint, RouteInfo, WeatherPanel, AccessNote, PlaceFacts, Recommendation, Dossier and the HitIDecision enumeration.
<code>catalog.py</code>	ACTIVITY_LABELS and <code>base_checklist()</code> encode the per-activity equipment baseline for the full catalogue (road, gravel and mountain cycling, hiking, trail running, tennis, padel, climbing, kayaking, cultural).
<code>intake.py</code>	<code>geocode()</code> via OpenStreetMap Nominatim; <code>parse_freetext()</code> turns a description into structured fields; <code>build_inputs()</code> assembles a validated ActivityInputs.
<code>llm.py</code>	<code>complete()</code> calls Groq (primary then fallback model) and finally local Ollama. Used only for understanding and drafting.
<code>embeddings.py</code>	<code>embed_query()</code> and <code>embed_documents()</code> with nomic-embed-text-v1; the model is loaded lazily and cached.
Module	Responsibility and key entry points
<code>clients/weather.py</code>	<code>get_weather()</code> returns a forecast within the reliable horizon, or a clearly-labelled ERA5 seasonal baseline beyond it (Open-Meteo, no key).
<code>clients/geo.py</code>	<code>access_notes()</code> queries Overpass for access tags and barriers and reports findings with an explicit certainty level.
<code>clients/route.py</code>	<code>resolve_route()</code> parses a user GPX (primary) or generates a route with OpenRouteService; includes a haversine fallback.
<code>clients/culture.py</code>	<code>place_facts()</code> uses Wikipedia geosearch and summaries to return grounded history and points of interest with citations.
<code>clients/nature.py</code>	<code>likely_species()</code> queries GBIF for nearby biodiversity records.
<code>stores/qdrant_store.py</code>	<code>ensure_collection()</code> , <code>upsert()</code> , <code>search()</code> using <code>query_points()</code> ; client pinned to server minor v1.12.4.
<code>stores/neo4j_store.py</code>	<code>ensure_constraints()</code> , <code>link_place_pois()</code> , <code>related_pois()</code> ; APOC available, enhanced schema off.
<code>stores/databricks_store.py</code>	<code>append_audit_event()</code> and <code>last_audit_hash()</code> with explicit column lists to avoid schema contamination.
Module	Responsibility and key entry points
<code>recommend.py</code>	<code>recommend()</code> maps activity, route and weather to an equipment checklist and risk flags with rationale. Deterministic and explainable. Scales water by duration and heat, and raises combined high-risk patterns.
<code>audit.py</code>	<code>record()</code> appends a hash-chained event to the Databricks Delta audit table, or to a local file when Databricks is not configured.
<code>dossier.py</code>	<code>render_pdf()</code> composes the dossier as a PDF with ReportLab, including a clear advisory disclaimer.

Module	Responsibility and key entry points
<code>mesh.py</code>	<code>publish_event()</code> emits governance and observability events to the Solace broker, best-effort and non-blocking. Uses <code>without_certificate_validation()</code> for the pilot TLS connection.
<code>agents/nodes.py</code>	The node functions <code>run_route</code> , <code>run_weather</code> , <code>run_access</code> , <code>run_culture</code> , <code>run_recommend</code> and <code>run_compose</code> . Names are kept distinct from state keys.
<code>agents/graph.py</code>	<code>build_graph()</code> compiles the LangGraph with a module-level <code>MemorySaver</code> and <code>interrupt_before</code> the review node; <code>run_until_review()</code> and <code>resume_after_review()</code> .
<code>agents/crew.py</code>	<code>write_narrative()</code> drafts the grounded narrative; <code>CrewAI</code> is imported on the main thread (in the portal) before any worker threads.
<code>portal/app.py</code>	The Streamlit portal: template and conversational intake, GPX upload, the draft view and the HITL decision console.

5. Data Pipelines

The Medallion notebooks run in Databricks and feed the application. They embody the architecture's principle that the deployed app stays light by precomputing embeddings in the pipeline.

Notebook	Role
<code>sherpa_01_bronze_ingest.py</code>	Captures raw open-data responses with source and timestamp into the bronze Delta table.
<code>sherpa_02_silver_curate.py</code>	Standardises the raw payloads into typed place and point-of-interest rows.
<code>sherpa_03_gold_embeddings.py</code>	Builds the gold place table, computes nomic embeddings, and upserts the place-fact vectors into Qdrant Cloud so the app embeds only the short user query at runtime.

The notebooks read their credentials from the Databricks secret scope created during setup, so no secrets are written into the notebooks.

6. The Human-in-the-Loop Workflow in Code

The orchestration runs to `hitl_review` and pauses, because the graph is compiled with `interrupt_before=["hitl_review"]` and a shared module-level `MemorySaver`. The portal presents the draft and offers the decisions defined by `HitlDecision`: approve, modify, regenerate a section, save as draft, reject, acknowledge a warning, share, and schedule a reminder.

On approve, `dossier.py` renders the PDF and the path is recorded. On modify, the reviewer's free-text note is captured and the draft is rebuilt with the note applied. Every decision calls `audit.record()` and `mesh.publish_event()`, so the action is both tamper-evidently logged and visible on the event mesh.

7. Streamlit Community Cloud Deployment and Exact Secrets

Deploy the repository to Streamlit Community Cloud with the app entry point set to `src/portal/app.py`, and a repository-root `requirements.txt` equal to `requirements-cloud.txt`. In the app settings, open Secrets and paste exactly the following keys, filling in your values. These mirror the local `.env` and are copied into the environment at startup by `load_streamlit_secrets()`.

Secret key	Value to provide
DEPLOY_ENV	cloud
GROQ_API_KEY	Your Groq API key
GROQ_MODEL_PRIMARY	llama-3.3-70b-versatile
GROQ_MODEL_FALLBACK	llama-3.1-8b-instant
EMBED_MODEL	nomic-ai/nomic-embed-text-v1
EMBED_DIM	768
NEO4J_URI	neo4j+s://XXXX.databases.neo4j.io
NEO4J_USERNAME	neo4j
NEO4J_PASSWORD	Your Aura password
NEO4J_DATABASE	neo4j
QDRANT_URL	https://XXXX.qdrant.io
QDRANT_API_KEY	Your Qdrant Cloud key
QDRANT_COLLECTION	place_facts
DATABRICKS_HOST	https://dbc-XXXX.cloud.databricks.com
DATABRICKS_TOKEN	Your Databricks token
DATABRICKS_HTTP_PATH	/sql/1.0/warehouses/XXXX
DATABRICKS_CATALOG	sherpa
DATABRICKS_SCHEMA	pilot
SOLACE_HOST	tcps://mr-connection-XXXX.messaging.solace.cloud:55443
SOLACE_VPN	Your message VPN
SOLACE_USERNAME	Your Solace client username
SOLACE_PASSWORD	Your Solace client password
MESH_ENABLED	true
AEMET_API_KEY	Your AEMET key
ORS_API_KEY	Your OpenRouteService key
OVERPASS_URL	https://overpass-api.de/api/interpreter
HTTP_USER_AGENT	SHERPA-pilot/0.1 (contact: you@example.com)

Note: the Solace host must use `tcps://` with port 55443, as established during setup. The deployed mesh connection uses an encrypted TLS transport; certificate-chain validation is

disabled for the pilot because the bundled native library does not resolve the intermediate chain.

8. Deployment and Run Order

1. Extract `SHERPA_App_Scripts.zip` into `/home/pruebas/Descargas` and run `bash scripts/20_deploy_app.sh`.
2. In Databricks, run the notebooks in order: 01, then 02, then 03, to build the gold table and upsert place-fact vectors into Qdrant Cloud.
3. Run `bash scripts/21_seed_stores.sh` to create the Qdrant collection and Neo4j constraints.
4. Local run: `bash scripts/22_run_local.sh`. Cloud run: deploy to Streamlit Community Cloud with the secrets in section 7.
5. Publish any code changes with `bash scripts/13_github_sync.sh "feat: application code"` from the Setup package.

9. Dependencies and Resource Notes

Local development uses the three isolated environments created during setup: `.venv-langchain` runs the portal, the orchestration, the stores and the classical models; `.venv-crewai` runs the crews; and `.venv-mesh` runs the faithful local mesh. Streamlit Community Cloud uses the single merged `requirements-cloud.txt`.

Resource note: the in-process embedding model is heavy for the free tier, which is why the place-fact vectors are precomputed in `sherpa_03_gold_embeddings.py` and only the short user query is embedded at runtime. If the free tier still struggles, the runtime query embedding can be moved to a hosted call.

10. Limitations and Next Steps

- AEMET is available to layer into the weather client for Spain; the pilot uses Open-Meteo as the dependable default that needs no key.
- Closure and access data is fragmented; the geo client reports findings with explicit certainty rather than asserting closures it cannot verify.
- The deployed mesh uses an encrypted TLS transport with certificate-chain validation disabled for the pilot; production should use a Solace-provided trust store or the Enterprise build.
- The narrative crew currently runs as a grounded single call; a richer multi-agent CrewAI crew can be enabled, importing on the main thread first.
- Next steps: seed a broader set of reference locations, add the classical model training notebook to complement the rules, and extend the activity-specific equipment rules with field feedback.

End of document. *This completes the pilot deliverables: Foundational Architecture, Pilot Setup, and the Application Code.*